

Git Integration with Jenkins

Streamlining Your CI/CD Pipeline with Powerful Version Control



Agenda

1 *Role of Git in Jenkins Pipelines*

Understanding the fundamental relationship

2 *Installation & Configuration*

Setting up Git on your Jenkins server

3 *Jenkins + Git Workflow*

From code commit to deployment

4 *Practical Implementation*

Freestyle jobs and webhook integration

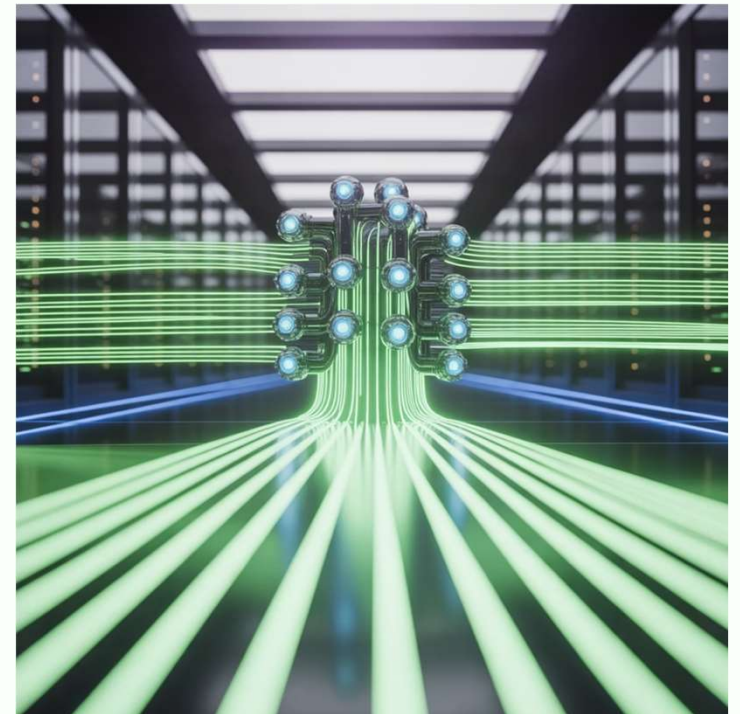
5 *Live Demonstration*

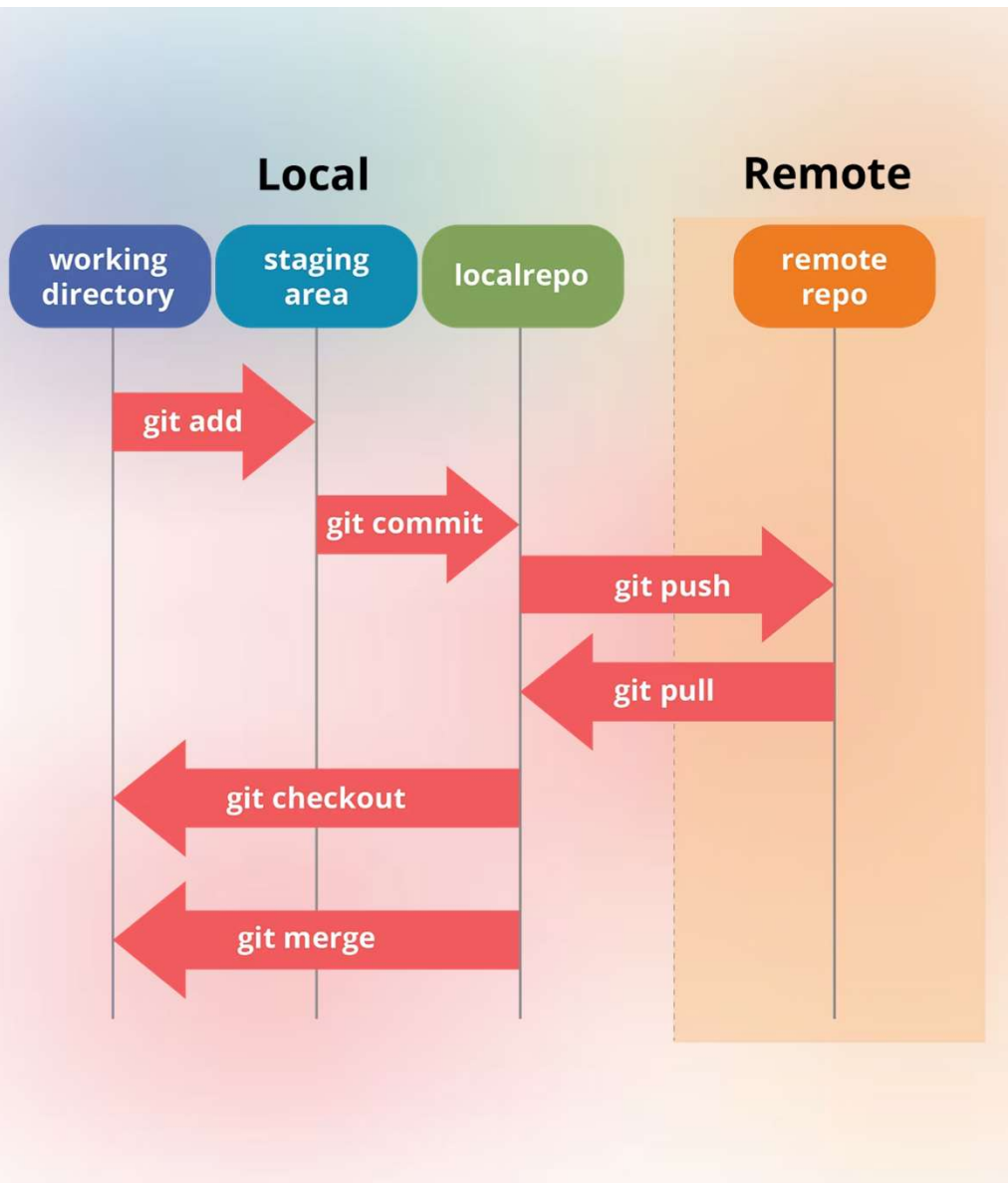
Seeing the integration in action

The Role of Git in Jenkins Pipelines

Git serves as the **foundation of modern CI/CD pipelines** within Jenkins by:

- Providing version control for application source code
- Enabling collaborative development across teams
- Triggering automated builds when code changes are detected
- Maintaining a historical record of all code changes
- Supporting branching strategies for feature development, testing, and production





Installing Git on Jenkins Server

Verify Java Installation

Ensure Jenkins is properly installed with compatible Java version

Install Git

For Linux: `sudo apt-get install git` or `sudo yum install git`

For Windows: Download and install Git from git-scm.com

Verify Installation

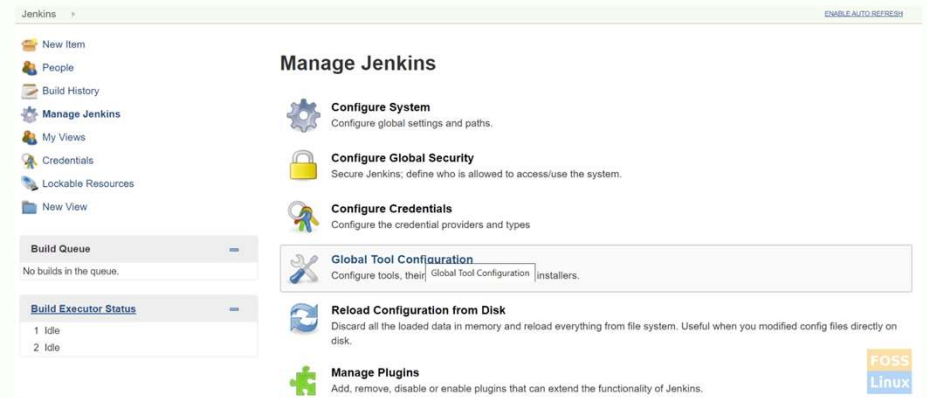
Run `git --version` to confirm successful installation

The Jenkins user must have appropriate permissions to execute Git commands. For Windows servers, ensure Git is added to the system PATH.

Configuring Git in Jenkins

Global Tool Configuration

1. Navigate to Jenkins Dashboard
2. Go to "Manage Jenkins" → "Global Tool Configuration"
3. Scroll to Git section
4. Click "Add Git"
5. Provide a name (e.g., "Default Git")
6. Specify path to Git executable
7. Click "Save"



The Global Tool Configuration allows Jenkins to locate and use the Git executable for all jobs requiring Git integration.

Jenkins + Git Workflow



Clone

Jenkins clones the Git repository to workspace



Build

Source code is compiled or packaged



Test

Automated tests verify code quality



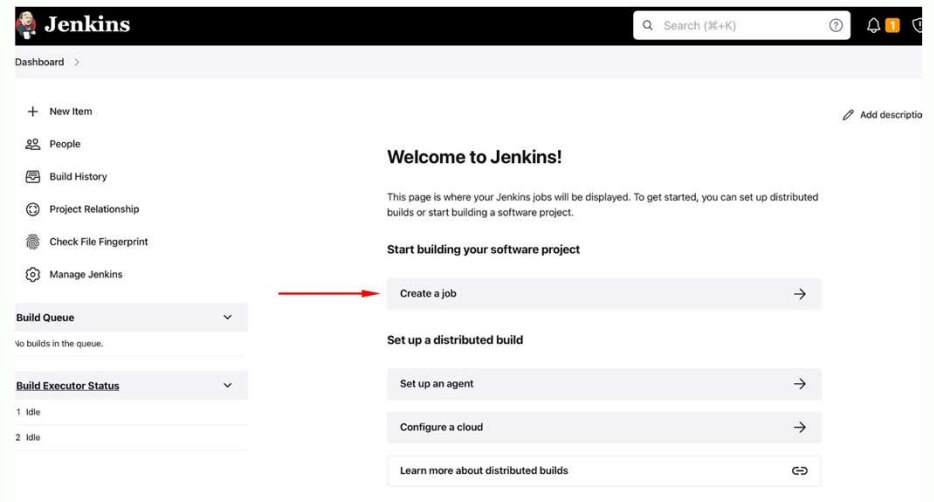
Deploy

Application is deployed to target environment

This workflow creates a **continuous integration and delivery pipeline** that automatically processes code changes from commit to deployment, ensuring consistent and reliable software releases.

Creating a Jenkins Freestyle Job with Git

1. From Jenkins dashboard, click "New Item"
2. Enter job name and select "Freestyle project"
3. In Source Code Management section, select "Git"
4. Enter repository URL (HTTPS or SSH)
5. Specify branch to build (e.g., */main, */master)
6. Configure build triggers (e.g., poll SCM)
7. Add build steps (e.g., Execute shell/batch command)
8. Save the configuration



Freestyle projects offer flexibility for simple integration scenarios and are an excellent starting point for Git integration with Jenkins.

Advanced Git Configuration in Jenkins

Credentials Management

Store Git credentials securely using Jenkins Credentials Plugin for accessing private repositories

Branch Specifiers

Configure specific branches or patterns (e.g., */feature/* for all feature branches)

Submodules

Enable "Advanced" options to recursively clone Git submodules

Shallow Clone

Optimize performance by limiting history depth for large repositories

These advanced configurations help tailor Git integration to your specific project requirements and optimize build performance.

Webhook Integration with Git Providers

Setting Up Webhooks

Webhooks enable **automatic triggering of Jenkins jobs** when code changes are pushed to your Git repository.

Supported Git Providers:

- GitHub
- GitLab
- Bitbucket



Projects Wiki Security Insights

Webhooks / Add webhook

We'll send a POST request to the URL below with details of format you'd like to receive (JSON, x-www-form-urlencoded [documentation](#)).

Payload URL *

Content type

Secret

Which events would you like to trigger this webhook?

☒ Just the push event.

☐ Send me **everything**.

☐ Let me select individual events.

GitHub Webhook Configuration

Jenkins Setup

Install GitHub Integration plugin in Jenkins

Configure job to use "GitHub hook trigger for GITScm polling"

GitHub Setup

Go to repository Settings → Webhooks → Add webhook

Set Payload URL to: `http://your-jenkins-url/github-webhook/`

Select "application/json" content type

Test Connection

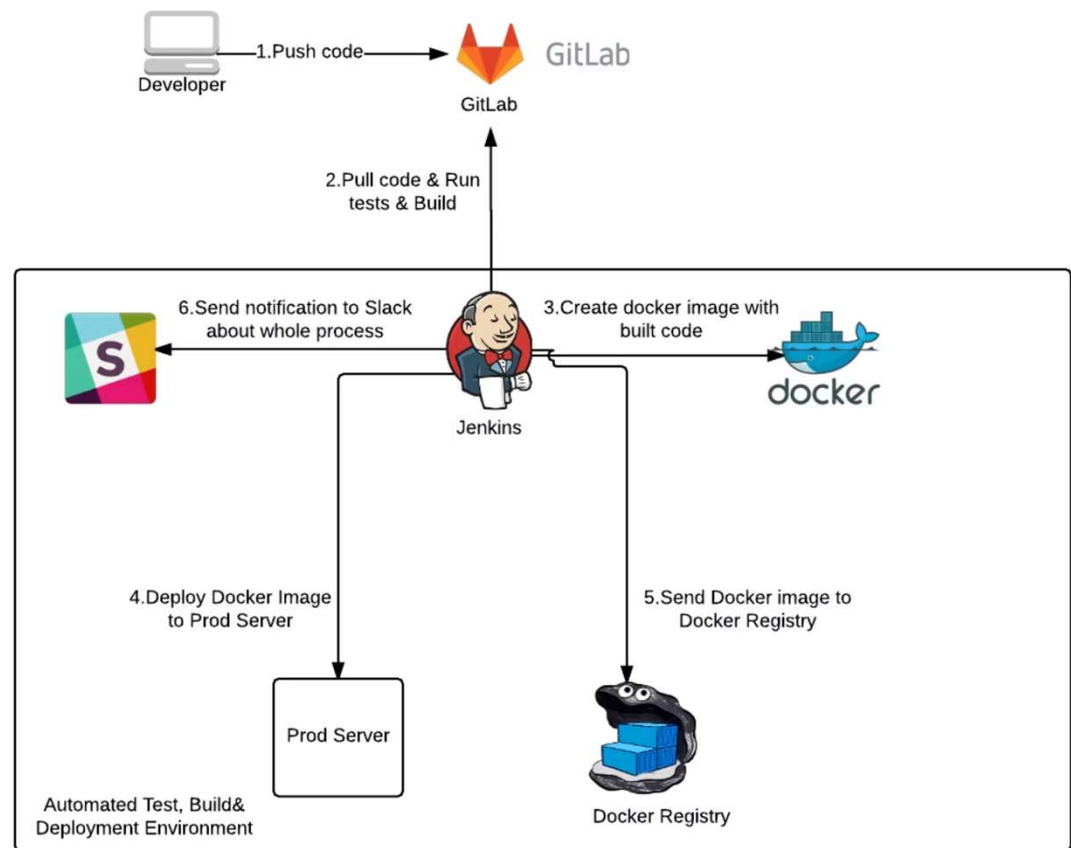
Click "Test" button in GitHub webhook settings

Verify Jenkins receives the webhook in system logs

Demo: Git Push to Jenkins Build

Workflow Demonstration

1. Developer commits code and pushes to Git repository
2. Git provider sends webhook notification to Jenkins
3. Jenkins job is automatically triggered
4. Build process executes (compile, test, package)
5. Build results are reported back to Git provider



This automated workflow eliminates manual build initiation, accelerating the development cycle and

Key Takeaways

Seamless Integration

Git and Jenkins form a powerful combination for modern CI/CD pipelines, automating the software delivery process.

Simple Setup

Installing and configuring Git with Jenkins requires minimal effort but delivers significant workflow improvements.

Automated Workflows

Webhooks eliminate manual intervention, creating a truly automated pipeline from code commit to deployment.

Next Steps

Explore Jenkins Pipeline as Code with Jenkinsfile stored in your Git repository for even more powerful integration.

Questions? Contact: devops@example.com